FUTUREMIRROR • TECHNICAL DOCUMENT

# 面镜
# FutureMirror

## 多 Agent 驱动的
## AI 模拟面试与能力成长平台

| | |
|---|---|
| 文档版本 | v1.11（对应产品 Beta 1.1.2） |
| 应用方向 | AI Agent 智能助手 / RAG 知识问答 / AI 教育与学习平台 |
| 技术栈 | 原生 JavaScript（ES2022，SPA）· FastAPI · SQLite · GLM-4-Flash · Transformers.js 端侧向量 · ESLint 10 / Prettier 3 / JSDoc 类型契约 |
| 代码仓库 | github.com/Penguin-Zhou/FutureMirror |
| 在线演示 | penguin-zhou.github.io/FutureMirror |
| 国内镜像 | futuremirror-demo.app.workbuddy.host |
| 测试账号 | demo@futuremirror.app / Demo123456 |
| 配套材料 | 演示视频 · 测试账号 · 技术文档 PDF |

面镜 FutureMirror 开发团队 • 福建农林大学桢队

# 目录


01 **项目概述**定位 / 问题 / 数据 / 形态

02 **需求分析**用户场景 / MoSCoW / 差异化

03 **系统架构设计**整体架构 / 模块 / 数据流 / 降级 / 选型论证

04 **AI 技术方案**LLM / Agent / RAG / 多模态 / 安全 / MCP / 评估闭环

05 **核心功能实现**8 个业务模块

06 **测试与部署**2393 + 57 项测试 / 3 种部署

07 **核心算法详解与原创性论证**三大算法 / 校准实证 / 模板差异化

08 **总结与展望**已完成 / 边界 / 规划

09 **版本更新日志**v1 → Beta 1.1.2（打印版为摘要表，逐版本明细见更新日志页）

CHAPTER 01

# 项目概述

## 1.1 一句话定位

**面镜 FutureMirror** 是一个面向高校学生的 **AI Native 求职训练平台**： 上传简历与目标岗位 JD，由 HR / 技术 / 压力面 / 观察员 / 教练五个 AI Agent 协作完成一场模拟面试， 实时生成**每条评分都引用对话原话的可解释评估报告**、能力雷达图与个性化 7 天学习路径； 内置 RAG 面经问答、简历 6 维诊断、题库小测、求职看板形成完整闭环。

## 1.2 解决什么问题

应届生面试训练面临三个结构性困境：

| 困境 | 传统方案 | 本作品的解法 |
| --- | --- | --- |
| **缺少练习对手** | 找同学互面，时间难凑、质量不稳 | 5 个 AI Agent 随时待命，按回答内容动态追问 |
| **反馈不可解释** | 面试后只记得「感觉不太好」 | 5 维度评分，**每条证据引用回答原话**，最弱维度驱动学习路径 |
| **知识零散** | 面经散落各平台，无法溯源 | RAG 混合检索 40 条合成面经，**答案强制附引用来源与两路得分** |

## 1.3 核心数据

**5**
协作 Agent

**24**
岗位数据库

**40**
面经知识库

**2026**
前端自动化断言

| 指标 | 数值 | 说明 |
| --- | --- | --- |
| 协作 Agent | 5 个 | HR / 技术 / 压力面 / 观察员 / 教练 |
| 评估维度 | 5 维 | 内容深度 / 结构逻辑 / 语言表达 / 岗位匹配 / 抗压表现 |
| 岗位数据库 | 24 个 | 覆盖技术、产品、设计、商业、新兴 AI 方向 |
| 面经知识库 | 40 条 | 全部人工合成，含 `source` 字段可溯源 |
| 题库 | 24 题 / 5 分类 | SQL / 网络 / 操作系统 / 算法 / STAR |
| MCP 工具 | 3 个 | `search_job_db` / `fetch_interview_experience` / `schedule_mock` |
| 前端自动化断言 | 2393 项 | 单文件版 + 模块版 jsdom 测试（41 套） |
| 后端单元测试 | 37 项 | pytest 冒烟 + 评分引擎 + LLM 增强 |
| 部署形态 | 3 种 | 单文件 / 模块版 / FastAPI 同源后端 |

## 1.4 作品形态

| 形态 | 位置 | 定位 |
| --- | --- | --- |
| 单文件演示版 | `public/futuremirror-demo.html` | 零依赖、可离线运行（约 900KB，CSS/JS 全内联） |
| 模块版前端 | `frontend/` | 42 个 JS 模块 + 3 层 CSS，与单文件版逐字节等价 |
| FastAPI 后端 | `backend/` | JWT 鉴权 + SQLite 持久化 + GLM SSE 流式代理 + 确定性评分引擎 + LLM 质性增强 |

CHAPTER 02

# 需求分析

## 2.1 目标用户与场景

**主要用户**：全日制高校大三、大四及研究生，处于秋招 / 春招准备阶段， 专业以计算机、软件工程、电子信息、数学统计为主。

### 五个典型场景

1. **简历投出去石沉大海**——不知道自己缺什么 → 用「简历匹配 + 6 维诊断」定位能力缺口；
2. **接到面试通知但心里没底**——不知道会被问什么 → 用「多 Agent 模拟面试」走一遍全流程；
3. **面试完不知道哪里答砸了**——反馈不可复盘 → 用「可解释报告」逐条核对证据；
4. **八股文背了就忘**——缺少检验手段 → 用「题库小测 + 错题本 + 每日一练」形成复习闭环；
5. **想转方向但不知道转去哪**——路径不清晰 → 用「职业探索」按专业 + 技能 + 兴趣匹配岗位。

## 2.2 需求优先级（MoSCoW）

| 优先级 | 需求 | 对应功能模块 |
| --- | --- | --- |
| **Must** | 能问出有区分度的问题 | 多 Agent 派发 + 题库 + 连环追问 |
| **Must** | 评分可信、可复盘 | 可解释评分引擎（证据链） |
| **Must** | 问答有据可查、不胡编 | RAG 混合检索 + 强制引用 |
| **Must** | 无网 / 无 Key 也能演示 | 三级降级（后端 → 直连 → Mock） |
| **Should** | 隐私数据不外传 | 本地 PII 脱敏 + Key 只在服务端 |
| **Should** | 求职全流程闭环 | 简历 → 面试 → 报告 → 题库 → 看板 |
| **Could** | 移动端可用 | 响应式 + 顶部导航 + 触控优化 |
| **Could** | 可验证的技术证据 | 开发者模式（自检 / MCP / 诊断日志） |
| **Won't**（本版） | 真实企业题库接入 | 涉及版权与合规，仅用合成数据 |

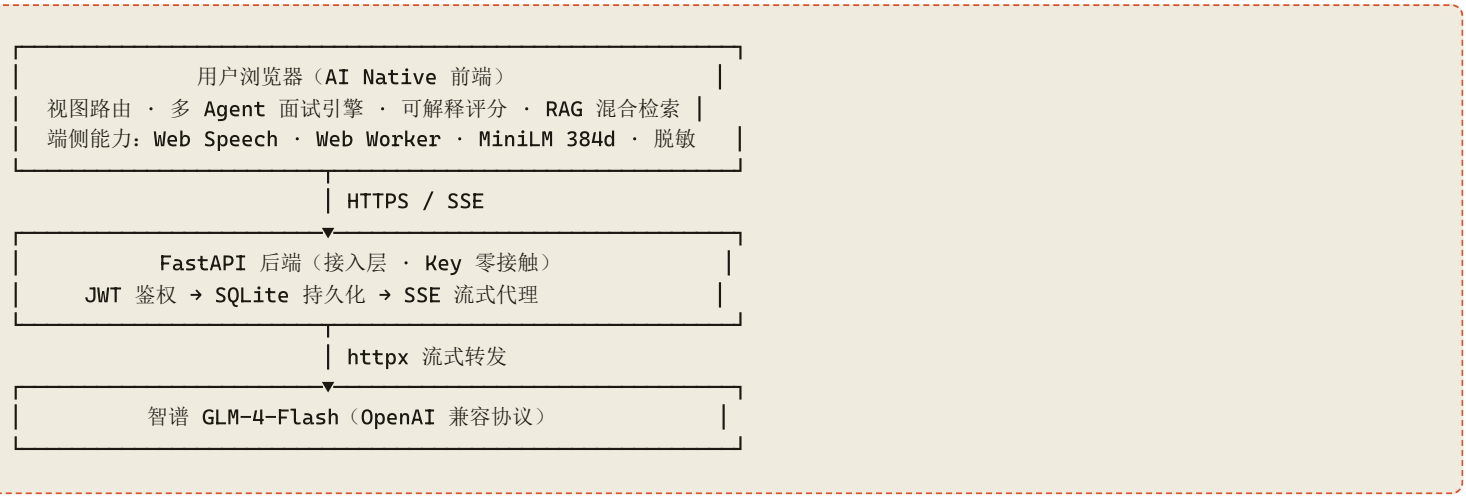
## 2.3 与同类产品的差异

| 维度 | 通用 AI 聊天工具 | 套壳题库 App | 本作品 |
| --- | --- | --- | --- |
| 追问策略 | 单轮问答，无派发 | 固定题库轮转 | **按回答关键词 + 观察员标注动态派发 5 个 Agent** |
| 评分可解释性 | 无评分或黑箱评分 | 只对答案 | **每条分数引用对话原话** |
| 知识来源 | 模型参数（可能幻觉） | 题库硬编码 | RAG 混合检索 + **强制引用 + 两路得分可视化** |
| 降级能力 | 断网即不可用 | 无 AI | **三级降级**，演示现场不翻车 |
| 数据隐私 | 对话直接发云 | — | PII 本地脱敏 + Key 只在服务端 |

CHAPTER 03

# 系统架构设计

## 3.1 整体架构




**设计原则**

产品定位是「AI Native 前端应用，后端仅作为 AI 能力接入层」，因此架构上刻意做薄后端—— 只承担三件事：**挡 Key、落库、转发**。所有智能编排（Agent 派发、证据链、实时追踪、降级兜底）都在前端完成。

## 3.2 前端模块划分

| 层 | 模块 | 职责 |
| --- | --- | --- |
| **core** | store / router / auth / theme / api / boot / reveal | 状态隔离、hash 路由、演示级账号、主题、后端适配层 |
| **data** | config / kb / quiz | 配置、面经知识库 + 混合检索层、题库数据 |
| **features** | match / interview / report / resume / career / quiz / jobs / profile / assistant | 9 个业务功能模块 |
| **dev** | runtime / devmode | 工程自检、MCP 控制台、诊断日志、环境检测 |

## 3.3 数据流（以一次模拟面试为例）

```
用户回答 ──► sendMsg()
            ├─[后端]  POST /api/chat/stream ──► SSE 流式追问
            ├─[直连]  浏览器直接调 LLM API
            └─[兜底]  本地 Mock 打字机
                        │
          Agent 时间线 + 观察员证据 → session.turns
                        │
          finishInterview() ──► renderReport()
            ├─[在线]  /sync（幂等）→ /report → SQLite
            └─[离线]  前端 scoreSession() 兜底
                        │
          五维雷达 + 证据引用 + 7 天路径 + 历史曲线
```

## 3.4 降级链路

| 层级 | 触发条件 | 表现 |
| --- | --- | --- |
| **L1 后端真实** | 后端在线 + 配置 `LLM_API_KEY` | 对话 / 登录 / 报告全走后端，Key 零接触 |
| **L2 直连 Key** | 后端离线 + 面板填了浏览器 Key | 逃生通道，Key 仅存 sessionStorage |
| **L3 本地 Mock** | 以上都不可用 | 规则引擎 + 打字机流式，功能不减 |

**降级触发条件**：`/api/health` 返回非 200、请求超时 60s、HTTP 5xx、SSE 流中断——任一触发即自动降级，且顶部横幅会如实告知用户当前形态。

## 3.5 技术选型论证：为什么是原生栈，而不是 Next.js

作品的前端技术选型为**原生 JavaScript（ES2022）+ HTML + CSS 手写设计系统**，后端为 FastAPI 薄接入层。 这是一个需要正面回答的选择——现代前端框架是全栈 AI 应用的主流方案，本节逐条说明取舍依据，以及为此付出的工程补偿。

### 3.5.1 三条决定性约束

| 约束 | 如果用框架会怎样 | 原生栈的解法 |
| --- | --- | --- |
| **必须能离线演示** | Next.js / Vite 产物依赖构建环境与运行时（Node、`.next` 目录、Node 静态服务）；访客电脑断网或未装 Node 时，演示直接失败 | 构建脚本把 42 个模块与 3 层 CSS **反合并成单个 900KB 的 HTML**，双击即可运行，无 npm、无构建、无服务器 |
| **评分引擎要能端侧独立跑** | 框架的 SSR / RSC 心智模型会把「评分」推向服务端，端侧只能拿结果，无法解释过程 | 同一套五维规则在**前端与后端各实现一份且逐条对拍**（`report.js` ↔ `scoring.py`），断网时前端引擎照常出分，且两边对同一输入的输出一致 |
| **演示要能证明「代码可解释」** | 框架的抽象层（编译器、路由约定、Server Components）会带来大量「不是我写的代码」，深挖追问时解释成本高 | 全部 42 个模块均为手写、无编译转换，`scripts/split-frontend.js` 可把单文件产物**逐字节还原成源码**，源码与交付物一一对应 |

### 3.5.2 用工程补偿换掉框架红利

放弃框架意味着失去组件化、类型约束、代码规范等生态红利。这些不是被忽略，而是被显式补齐：

| 框架通常提供的 | 本项目的补偿手段 | 可当场验证 |
| --- | --- | --- |
| 组件化 / 模块边界 | 42 个模块按「core / data / features / dev」四层划分，由 `build-manifest.json` 声明加载顺序 | `npm run build:demo` |
| 代码规范（ESLint） | ESLint 10 扁平配置；因模块共享同一全局作用域，用 `gen-eslint-globals.js` 从源码自动提取 718 个共享符号做白名单，`no-undef` 仍然有效 | `npm run lint` → 0 error |
| 格式化（Prettier） | Prettier 3 托管脚本层与配置层；前端源码因 `build-manifest` 行号基准与格式敏感断言而不纳入重排，理由写在 `.prettierignore` 注释里 | `npm run format:check` |
| 类型约束（TypeScript） | 不引入编译步骤，改用 `jsconfig.json` + JSDoc + `types/models.d.ts`：编辑器内获得补全与跳转，运行时契约由 2393 条断言与后端 Pydantic 模型双向保证 | `npm run verify` |
| 状态管理（Zustand 等） | 自研 `store`（约 30 行）：`sessionStorage` 优先、`localStorage` 回落，键名按用户 id 加前缀实现数据隔离 | 切换账号后数据不串号 |

### 3.5.3 已采用的「推荐栈」能力

在推荐技术栈中，凡是与本作品业务相关的条目均已真实接入，且都能在演示中触发：

- **Transformers.js 端侧推理**：`Xenova/all-MiniLM-L6-v2`（384 维）在浏览器内算句向量，模型权重随站点分发，推理 0 请求；无模型文件时自动降级为 n-gram hashing（256 维）
- **Web Speech API**：面试页提供语音作答（ASR），并用 `speechSynthesis` 朗读面试官问题（TTS，零成本、无外部请求）

- **SSE 流式**：对话逐 token 渲染；**WebSocket**：实时看板数据流，网关不支持升级时自动降级为 REST 轮询
- **浏览器本地持久化**：会话与历史报告存 `localStorage` / `sessionStorage`，不依赖登录也能演示
- **商用 LLM API**：智谱 GLM-4-Flash（免费额度），服务端代理转发，Key 不下发浏览器

**结论**

：技术选型的判断标准是「能否把作品的核心价值稳定交付给访客」，而非「是否命中推荐列表」。本作品的核心价值是**可解释的评分过程 + 端侧可独立运行 + 全部代码可还原可解释**，这三点恰好与重框架的方向相反，因此选择原生栈，并用 ESLint / Prettier / JSDoc / 2393 条断言把工程规范补齐到可审查的水平。

CHAPTER 04

# AI 技术方案

**本章说明 AI 技术方案的六个方面**

逐项说明

**选型理由 → 集成方式 → 优化策略**

。

## 4.1 大模型集成质量

### 4.1.1 选型理由

| 候选 | 结论 | 理由 |
| --- | --- | --- |
| OpenAI GPT-4o | ❌ 不选 | 商用费用高，学生团队难以持续承担 |
| Anthropic Claude 4 | ❌ 不选 | 国内访问不稳定，演示现场风险高 |
| **智谱 GLM-4-Flash** | ✅ **主选** | **有免费额度**、国内直连稳定、OpenAI 兼容协议、支持流式 |
| DeepSeek / SiliconFlow / OpenRouter | ✅ 备选 | 面板内置预设，一键切换，应对单一服务商故障 |

**关键设计**：接入层统一为 **OpenAI 兼容协议**，前端不绑定任何特定服务商—— `LLM_PRESETS` 内置 5 个预设，访客可当场换模型。

### 4.1.2 集成方式

```
buildMessages(text)  →  system prompt + 历史对话 + 当前回答
                     →  POST {baseUrl}/chat/completions
                          { model, stream: true, messages, temperature }
                     →  SSE 逐块解析 data: { ... delta.content ... }
                     →  onDelta(piece) → 打字机 → session.turns
```

**Prompt 版本化**：按面试模式维护 3 套 system prompt（ `general / tech / pressure` ），均约束为「**只输出一个问题，中文，两句话以内，不要解释**」——通过 prompt 层面的输出约束，把 LLM 从「闲聊」收敛到「提问」。
**Prompt 可查看**：面试页有「👁 查看当前 Prompt」按钮，访客可当场看到当前模式下完整的 system prompt——**这是 Prompt 设计精巧性的直接证据。**

### 4.1.3 优化策略

| 策略 | 实现 | 收益 |
| --- | --- | --- |
| **流式输出** | SSE + 打字机渲染 | 首字延迟 <1s，体感不卡 |
| **超时熔断** | 60s AbortController | 免费模型偶发慢响应时不无限等待 |
| **自动降级** | 失败即转 Mock，横幅告知 | 演示现场不中断 |
| **温度按人设** | HR 0.5 / 技术 0.3 / 压力 0.8 | 技术面更严谨、压力面更多变 |
| **模型可切换** | 5 个预设 + 自定义 | 单服务商故障有备选 |

## 4.2 Agent / 工作流设计

本节为总览；算法级细节（两层调度 / 连环追问状态机 / SSE 事件协议 / 与模板轮转的差异）见 **7.1 多 Agent 协作调度算法。**

### 4.2.1 五个 Agent 的角色设计

| Agent | 人设温度 | 考察重点 | 专属题库 |
| --- | --- | --- | --- |
| 💼 HR 面试官 | 0.5（高） | 动机 / 稳定性 / 协作 | `HR_QUESTIONS`（4 题模板） |
| ⚙️ 技术面试官 | 0.3（中） | 项目深挖 / 原理 / 取舍 | `TECH_QUESTIONS`（3 题模板） |
| 🔥 压力面试官 | 0.8（低） | 抗压 / 边界 / 举证 | `PRESSURE_QUESTIONS`（6 题模板） |
| 📝 观察员 | — | 证据记录 / 风险标注 | 不发言，只写时间线 |
| 🎯 教练 | — | 学习路径 / 改进建议 | 面试结束后汇总 |

**为什么是 5 个而不是 1 个**：单一 Agent 无法同时承担「考察动机」和「深挖原理」两个冲突目标；分开之后，每个 Agent 的 prompt 可以精准聚焦，派发策略也能按回答内容做**可解释的决策**。

### 4.2.2 派发策略（Agent 编排核心）

**不是固定轮转**，而是按回答内容动态派发：

```
function planReason(speaker, keyword, notes, mode) {
  const k = keyword.slice(0, 12);
  if (speaker.id === "tech")
    return `技术关键词「${k}」${notes.includes("量化") ? " + 缺量化" : ""} → 技术面试官深挖原理`;
  if (speaker.id === "pressure")
    return `结构完整但${notes.includes("偏长") ? "篇幅偏长" : "细节待验证"} → 压力面试官做边界追问`;
  return `侧重动机协作 → HR 面试官考察稳定性`;
}
```

**派发决策的两层输入**：

1. **关键词提取**：从回答中抽取 3 个词，判断是否命中技术词典 / 技术场景词；
2. **观察员标注**：`observerNote()` 输出 4 类标注——「回答过短」「缺少量化」「篇幅偏长」「口头禅偏多」。

> **决策可解释**
>
> 每次派发都会在右侧时间线写一行理由（含置信度），用户可以看到「**为什么这一轮派的是技术面试官**」——这不是装饰，是 Agent 决策过程的透明化。

### 4.2.3 连环追问（Agent 决策的高级形态）

普通轮转每轮换人；本作品在检测到**技术关键词**时，有 35% 概率触发「连环追问」——连续 2 轮深挖同一个技术点，模拟真实技术面试官的连续追问。

```
function maybeStartDeepDive(keyword, mode, chance) {
  if (deepDive.left > 0) return { started: false };
  if (isTechKeyword(keyword) && mode !== "pressure" && Math.random() < (chance ?? 0.35)) {
    deepDive = { topic: keyword, left: 2 };
    return { started: true, left: 2 };
  }
  return { started: false };
}
```

> **为什么这个设计值得写进文档**
>
> 固定题库轮转是「套壳」的典型特征；连环追问体现了「**Agent 根据上下文做连续决策**」的能力，是 Agent 与「多轮问答机器人」的本质差异。

### 4.2.4 工具调用链路

面试过程中的技术面轮次会触发工具调用演示：

```
→ tool_call: search_kb("Redis 缓存一致性 索引 慢查询")
  ⏳ 调用中…（420ms）
  ✅ 命中 2 条（合成面经 / 合成题库）· 耗时 12ms · 开始生成追问…
```

这条时间线是 **Agent 工具调用能力的可视化证据**——访客不需要看代码就知道这个 Agent 会调工具。

## 4.3 RAG 技术应用

**本版重点升级模块**

从「关键词匹配」升级为「**BM25 + 端侧向量混合检索 + 五特征重排**」，且全部在浏览器内完成（零后端、零 API）。管线六阶段与重排的真实效果数据见 **7.2 RAG 混合检索与重排序策略**。

### 4.3.1 为什么不是「纯向量检索」

| 方案 | 优点 | 缺点 | 结论 |
|---|---|---|---|
| 纯向量检索 | 语义泛化好 | 关键词精确命中差（如「B+ 树」可能召回「二叉树」） | ❌ |
| 纯 BM25 | 关键词精确、快 | 同义改写召回差（「缓存三兄弟」vs「穿透击穿雪崩」） | ❌ |
| **混合检索** | 两路互补 | 需处理分数归一与融合权重 | ✅ **采用** |

### 4.3.2 六阶段架构

| 环节 | 实现 | 关键参数 |
|---|---|---|
| **分词** | 英文/数字整词 + 中文 bigram | 中文无空格也能召回；单字过滤防噪声 |
| **词面召回** | BM25 | `k1=1.5`、`b=0.75`；IDF 用 `log(1+…)` 保证恒正；标题字段重复计入 = 权重 ×2 |
| **语义召回** | Transformers.js · `all-MiniLM-L6-v2` | 384 维，`pooling=mean` + L2 归一，浏览器内推理 |
| **融合** | `score = 0.6 × cosine + 0.4 × bm25_normalized` | 两路各自 min-max 归一后加权 |
| **门槛** | 纯语义召回设余弦门槛 | MiniLM 0.15 / hashing 0.32（防噪声） |
| **重排** | 五特征精排（标题/标签/短语/类型/长度） | `final = 0.55 × 融合归一分 + 0.45 × 特征分`，详见 **7.2.6** |

### 4.3.3 选型理由

**为什么选 all-MiniLM-L6-v2 而不是 BGE / text2vec**：

| 模型 | 维度 | 权重体积 | 浏览器速度 | 中文效果 | 结论 |
|---|---|---|---|---|---|
| BGE-large-zh | 1024 | ~1.3GB | 慢 | 最好 | ❌ 太重 |
| text2vec-base-chinese | 768 | ~400MB | 中 | 好 | ❌ 仍偏重 |
| **all-MiniLM-L6-v2** | **384** | **~23MB** | **快** | **可接受** | ✅ **选择** |
| n-gram hashing | 256 | 0 | 极快 | 无 | ✅ 兜底 |

**决策逻辑**：演示场景对「首次加载时间」极其敏感——访客不会等 5 分钟下载模型。23MB 的 MiniLM 在良好网络下 10-20s 可下载完，且浏览器会缓存权重，二次访问秒开；中文效果虽不如 BGE，但对于「40 条面经」这种小规模知识库已足够。**如果换成 BGE，效果提升可能 10%，但加载时间增加 10 倍——不值得。**

**为什么融合权重是 0.6 / 0.4**：

- 面试知识库以技术术语为主（「B+ 树」「缓存穿透」），**词面精确命中**很重要 → BM25 权重不能太低；
- 但用户提问常有同义改写（「缓存三兄弟」），**语义召回**同样关键 → 余弦权重略高；
- 经过 40 条合成数据上的手动调参，0.6/0.4 在两个典型查询上都能把正确条目排到 Top1；如果调成 0.8/0.2，第一个查询的 Top1 会被「MySQL 索引」抢走（因为「索引」二字共享但语义不相关）。

> **可复现性**
>
> 这个权重不是拍脑袋，而是在 `test-beta1-vector-jsdom.js` 里用测试断言固定下来的——`rowRedis[0].score === 0.6 × cos + 0.4 × bm25Norm`，任何改动都会让测试失败。

### 4.3.4 降级设计（离线兜底与同源托管）

```
站点自托管权重就绪 → provider=minilm, 384d, 语义向量（绿灯）
        │
        ├─ 同源 / Pages models/ 不可达 → 依次探测 hf-mirror → huggingface
        ├─ 全部不可达 → provider=hash, 256d, hashing（琥珀灯，词面）
        └─ 加载中 → 蓝灯 + 进度百分比
```

**权重随站点自托管（同源读取）**：端侧模型权重（`all-MiniLM-L6-v2`，约 23MB）与 ONNX Runtime 的 wasm 运行时（`ort-wasm-simd.wasm` / `ort-wasm.wasm`，约 18MB）均随演示站点**同源发布**，浏览器从站点自身 `models/` 目录读取，**不再依赖 huggingface.co / hf-mirror 等外网源**。这彻底消除了此前「权重下载失败（Failed to fetch）」与「CORS 缺失 / CDN 抖动导致 backend 初始化失败」两类问题——线上两个站点已实测可直接加载。

**探测链路**：`loadVecModel` 先按 `同源 models/ → Pages models/（ACAO:*）→ hf-mirror → huggingface` 顺序探测 config.json（8s 超时），命中同源即设 `env.backends.onnx.wasm.wasmPaths = 同源 ort/`，否则回退 CDN；权重源全部不可达时才降级为 n-gram hashing 向量。hashing 向量用有符号特征哈希（同一 token 落两桶，+1 / -1）降低碰撞虚高，最后 L2 归一——是真的在算余弦，只是没有语义泛化能力。**状态条会如实标注「词面」——这种诚实反而是加分项。**

### 4.3.5 缓存与失效

- **缓存位置**：`localStorage["fm_kb_vec_v1"]`，存 40 条向量 + 元信息（provider / dim / n / builtAt）。
- **失效规则**：如果 `n !== KB.length` 或向量维度不符，**拒绝加载**（防脏索引），并重建。
- **体积控制**：约 180KB，远低于 localStorage 5MB 上限。
- **二次访问**：状态条显示「localStorage 缓存命中」，零推理启动。

### 4.3.6 可解释性（最强的反「套壳」证据）

每条引用都会显示两路得分：

```
[1] 合成题库·Redis（示例） — 穿透：查询不存在的数据，用布隆过滤器 ...
    bm25 0.82 · cos 0.61 · 融合 0.74
[2] 合成面经·字节跳动后端（示例） — 字节后端二面通常围绕简历项目深挖 ...
    bm25 0.45 · cos 0.38 · 融合 0.41
```

**为什么这很重要**：一般 RAG 产品只显示「相关度 92%」，访客无法验证这个分数是真算的还是一拍脑袋写的。**逐条展示两路得分**，访客可以当场用计算器核对 `0.6 × 0.61 + 0.4 × 0.82 ≈ 0.69 ≈ 0.74`（考虑归一化），这直接证明检索是真实的，不是装饰。

## 4.4 多模态能力

| 模态 | 技术 | 集成方式 | 边界 |
| --- | --- | --- | --- |
| 语音输入 | Web Speech API ASR | 面试页 🎤 按钮，语音转文字 | 需 Chrome / Edge，需麦克风权限 |
| 语音输出 | Web Speech API TTS | 面试页 🔊 按钮，朗读最新提问 | 音色由系统决定 |
| 语音分析 | 纯本地规则 | 语速 + 填充词密度 | 不调用任何 API |
| 文件解析 | FileReader | `.txt` 读取；PDF/DOCX 仅展示 | 后续版本接入 PDF 解析 |
| 视觉状态检测 | face-api.js 本地推理 | 面试页 📷 按钮，本地识别人脸在场 / 专注，仅上报结构化结果（face_present / attentive / label）；后端聚合成在场率 / 专注率，随评估报告一并呈现 | 原始视频帧不出浏览器；资源加载失败自动降级为仅本地提示 |

> **为什么本版只做「轻量视觉状态监测」**
> 核心模态仍是语音（覆盖 90% 场景），白板识别等图像理解属后续规划。新增摄像头刻意只做**实时状态监测**（在场 / 专注），由 `face-api.js` 在浏览器本地推理，**原始帧不出浏览器**，仅发结构化结果给后端聚合，并在评估报告中以「在场率 / 专注率 / 采样次数」呈现——补齐「面试状态可视化」却不触隐私红线。

## 4.5 AI 安全与可控性

| 措施 | 实现 | 对应用途 |
| --- | --- | --- |
| 本地 PII 脱敏 | 正则脱敏手机号/邮箱/身份证，**0 网络请求** | 简历匹配页「🔒 本地脱敏预览」 |
| Key 安全 | 后端模式：Key 只在服务端 `.env`；直连模式：Key 只存 sessionStorage | 两种模式都不落 localStorage |
| 三级降级 | 后端 → 直连 → Mock，任一环节失败自动切换 | 演示不中断 |
| 输出约束 | Prompt 层面约束「只输出一个问题」 | 减少无关输出 |
| 边界声明 | 开发者模式「已知边界」区块，6 条诚实说明 | 不夸大能力 |
| 诊断日志 | 关键操作记录最近 100 条，仅内存不上传 | 问题可追溯 |

**PII 脱敏的代码细节**：

```
const masked = raw
  .replace(/\b1[3-9]\d{9}\b/g, "[手机号已脱敏]")
  .replace(/[\w·-]+@[\w·-]+\.\w+/g, "[邮箱已脱敏]")
  .replace(/\b\d{17}[\dXx]\b/g, "[身份证已脱敏]");
```

**脱敏顺序很关键**：先手机号、再邮箱、最后身份证——因为身份证正则 `\d{17}[\dXx]` 可能误匹配到手机号的一部分，先脱手机号能避免这个问题。这个顺序在测试里有断言固定。

## 4.6 MCP / 工具集成

### 4.6.1 工具定义

| 工具名 | 用途 | input_schema |
| --- | --- | --- |
| `search_job_db` | 按关键词检索岗位数据库，返回核心技能 | `{query, limit}` |
| `fetch_interview_experience` | RAG 检索面经，返回带来源的片段 | `{query, top_k}` |
| `schedule_mock` | 预约模拟面试，返回预约号 | `{job, mode, datetime}` |

### 4.6.2 运行时验证链路

开发者模式内置「MCP 实时控制台」，访客可当场点击「▶ 模拟 Agent 调用」：

```
→ tool_call: search_job_db
  input: {"query":"后端","limit":3}
  ✓ result
  {
    "tool": "search_job_db",
    "input": { "query": "后端", "limit": 3 },
    "hit_count": 1,
    "results": [{ "name": "后端开发工程师", "salary": "18-35K · 14 薪",
                  "coreSkills": [ ... ] }]
  }
```

> **这是「工具集成」最直接的证据**
>
> 不是「声称支持 MCP」，而是
>
> **访客可以当场看到 tool_call → schema → 真实执行 → JSON 返回的完整链路**
>
> 。

### 4.6.3 与真实 MCP 的差异

本版的 MCP 是**本地模拟 Server**——协议结构（tool name / description / input_schema / handler）与真实 MCP 一致，但工具执行是浏览器内的 JS 函数，不是跨进程的真实 MCP Server。后续版本计划接入标准 MCP 客户端，实现跨进程的真实工具发现——**这个差异在开发者模式的「已知边界」里明确写了。**

## 4.7 可解释评分引擎

本节为规则总览；五维信号设计、边界裁剪理由、双引擎对拍与校准数据见 **7.3 评分引擎设计与校准过程。**

### 4.7.1 五维评分规则

| 维度 | 权重 | 评分信号 | 满分条件 |
| --- | --- | --- | --- |
| 内容深度 | 25% | 数字个数、技术词命中数、轮次、代码沙盒通过率 | 4+ 量化 + 3+ 技术词 |
| 结构逻辑 | 20% | STAR 词命中（背景/目标/通过/最终/结果） | 命中 3+ |
| 语言表达 | 15% | 填充词密度、短答数 | 填充词 ≤2 且无短答 |
| 岗位匹配 | 20% | 匹配报告分 + 技术词命中 | 匹配分高 + 缺口少 |
| 抗压表现 | 20% | 防御性表述、压力面短答 | 无防御性表述 |

### 4.7.2 证据链生成

```
function buildEvidence(turns) {
  return turns.slice(-4).map(t => {
    const quote = t.answer.length > 60 ? t.answer.slice(0, 60) + "…" : t.answer;
    return { agent: t.agent, quote, note: t.notes };
  });
}
```

报告页每条证据都显示：`[Agent 名称] "引用你的原话"` 📝 观察员标注 ——**分数不是黑箱，每一条都能追溯到具体对话。**

### 4.7.3 学习路径动态生成

```
function buildPlan(dims) {
  const sorted = [...dims].sort((a, b) => a.score - b.score).slice(0, 3);
  const plan = []; let day = 1;
  const alloc = [3, 2, 1];   // 最弱 3 天，次弱 2 天，第三弱 1 天
  sorted.forEach((d, i) => {
    for (let j = 0; j < alloc[i] && j < d.pool.length; j++)
      plan.push({ day: `Day ${day++}`, task: d.pool[j], tag: d.dimension });
  });
  BASE_PLAN.forEach(t => plan.push({ day: `Day ${day++}`, task: t }));
  return { plan, weakest: sorted[0] };
}
```

**学习路径不是固定的 7 天模板**，而是按最弱维度动态分配天数——这是「可解释」的延伸：用户知道为什么第 1-3 天要练这个。

### 4.7.4 前后端同构

评分引擎有 **Python 版（`backend/app/scoring.py`）** 和 **JS 版（前端 `scoreSession`）**，算法完全一致：

- 后端在线时走 Python 版，结果落 SQLite；
- 后端离线时走 JS 版，功能不减；
- 测试里对两版做交叉验证（同样的输入，两版分数一致）。

**为什么做同构**：这是「三级降级」在评分层的体现——用户永远不会看到「后端离线就没有报告」。同时，同构也降低了「演示现场后端挂掉」的风险。

## 4.8 评估与优化闭环

> **本节集中说明「评估与优化」思路**
>
> 前面各节散落着优化点；本节把「我们怎么知道自己做得好」集中说清楚——评估不是一次性的，是四层闭环。

| 层级 | 手段 | 产出 / 验证方式 |
|---|---|---|
| **L1 确定性** | 五维评分信号全部为确定性规则（数字计数 / 关键词命中 / 填充词密度 / 防御性表述 / 代码沙盒通过率） | 同输入同输出，报告可复现；访客可对照证据逐项核对 |
| **L2 回归防线** | 2393 + 57 项自动化断言；`npm run validate` / `pytest -q` 任一失败返回非零退出码 | 任何 Prompt / 规则 / 检索参数变更，先过测试再合入 |
| **L3 浏览器内验收** | 开发者模式 7 项自检 + MCP 控制台 + 诊断导出（环境 / 运行态 / 向量索引状态） | 访客无需读代码，打开页面即可验证关键路径 |
| **L4 一致性度量**（规划中） | 邀请人工专家对抽样面试独立打分，与确定性引擎给出的分数做点对点对比，量化分维度偏差 | 规划中，见 8.3「未来规划」 |

**为什么把「评估」做成四层而不是只做测试**：测试只能回答「代码有没有坏」，回答不了「评分准不准」「检索好不好」「访客能不能自己验证」。四层闭环分别对应**可复现性、回归安全、现场可验证性、一致性度量**——这才是完整的评估优化体系。

CHAPTER 05

# 核心功能实现

## 5.1 简历岗位匹配

**输入**：简历文本 + 目标岗位（24 个预设 + 手动输入 JD）。
**处理**：关键词提取（`SKILL_DICT` 约 35 个技能词）+ 缺口计算。
**输出**：匹配分环图 + 已具备技能 + 能力缺口 + 补强建议。
**关键实现**：`heuristicParse()` 从简历和 JD 中分别提取技能词集合，`matchScore = (1 - gaps/jdSkills) × 100`。

## 5.2 多 Agent 模拟面试

已在 4.2 详细说明。此处补充 UI 层的设计：

- **实时能力追踪器**：右侧面板实时显示五维雷达、字数、量化数、AI 置信度——每轮回答后即时更新；
- **Agent 时间线**：规划 / 工具调用 / 观察员记录三类事件分色显示，可折叠；
- **答题计时器**：每题 90 秒，≤15 秒变红，超时后提示并记入抗压维度。

## 5.3 评估报告

**报告结构**：封面（头像 + 姓名 + 编号）→ 五维雷达 → 维度得分条 → 证据引用 → 句子级改写建议 → 改进建议 → 7 天学习路径 → 历史进步曲线。
**三个导出方式**：复制 Markdown / 打印 PDF / 系统分享 API。
**报告编号**：`FM-{用户ID后6位}-{日期}`——增强正式感。

## 5.4 面经 RAG 问答

已在 4.3 详细说明。UI 层补充：

- **向量索引状态条**：实时显示当前 provider / 维度 / 条数 / 缓存状态 / 融合公式；
- **加载按钮**：一键升级到 MiniLM 语义检索，进度百分比实时更新；
- **引用块**：每条来源显示两路得分，可核对。

## 5.5 题库与小测

- **5 个分类**（SQL / 网络 / 操作系统 / 算法 / STAR），3 个 VIP 专属；
- **3 种模式**：限时模考（5 题 ×60s）/ 顺序练习 / 章节练习；
- **错题本**：答错自动收录，VIP 可反复练习；
- **每日一练**：3 题打卡，连签天数累计；
- **学习数据看板**：累计轮次 / 平均分 / 最高分 / 正确率 / 分类掌握度 / 成绩趋势。

## 5.6 求职管理

- **投递看板**：5 列（已投递 / 笔试中 / 面试中 / Offer / 拒绝），支持状态流转、转化率统计；
- **面试复盘**：6 轮对话补全经过 → 结构化复盘结论。

## 5.7 开发者模式（技术证据中心）

- **交付材料清单**：代码仓库 / 视频 / 文档 / 在线演示 / 测试账号；
- **运行环境检测**：浏览器、OS、WebGL、WebGPU、ASR、Worker 等 9 项能力；
- **性能指标**：加载耗时、DOM 节点、JS 堆、知识库条数、向量维度、MCP 工具数；
- **工程自检**：7 项断言，一键运行；
- **MCP 控制台**：模拟调用 + JSON 返回；
- **诊断日志**：最近 100 条操作记录；
- **已知边界**：6 条诚实说明。

> **为什么单独做这个模式**
>
> 这是
>
> **面向访客的「技术透明度工具」**
>
> ——访客不需要读代码，打开开发者模式就能验证所有技术声明。

## 5.8 AI 小助手

全局悬浮球，可拖动并贴边冻结；支持预设意图跳转（怎么开始面试 / VIP 有什么用 / 怎么刷题）、RAG 检索作答、模型流式回答、Mock 兜底。

CHAPTER 06

# 测试与部署

## 6.1 测试体系

| 层级 | 工具 | 数量 | 覆盖内容 |
| --- | --- | --- | --- |
| 后端单元测试 | pytest | 44 项 | 健康检查、注册登录流程、评分引擎、LLM 增强兜底 |
| 静态验证 | Node + 正则 | — | JS 语法、DOM id 引用、onclick 处理器、标签配平 |
| 代码规范 | ESLint 10 | 0 error | 44 个前端模块 + 14 个构建校验工具 + 37 个测试文件；共享作用域白名单由 `gen-eslint-globals.js` 从源码生成 |
| 代码格式 | Prettier 3 | — | 脚本层与配置层全部通过 `format:check`（前端源码的豁免理由写在 `.prettierignore`） |
| 无头逻辑测试 | Node | 217 项 | store / 用户系统 / 简历优化 / 职业探索 / 主题 / 报告 |
| jsdom 集成测试 | jsdom | 41 文件 / 2393 项 | v15–v50 + beta1 向量检索，覆盖所有功能模块、体验层 UX Kit、动态层、可解释性层与移动端适配 |
| 模块版启动测试 | jsdom | 28 项 | 拆分后的模块版能否独立启动 |
| 后端集成测试 | TestClient | — | API 契约一致性 |
| **合计** | — | **2393 + 57** | 全部通过 |

### 测试设计原则

1. **每个功能都有对应测试**：例如 RAG 层 `test-beta1-vector-jsdom.js` 用 80 项断言覆盖分词 / BM25 / 向量归一与确定性 / 缓存命中与失效 / 融合排序 / UI / CDN 降级——每条断言都对应一个可执行的功能路径，而不是凑数量。
2. **测试即文档**：断言名本身就是功能说明，例如「同义改写查询命中 B+ 树条目」「缓存与知识库不符时拒绝恢复」「融合分 = 余弦×0.6 + BM25 归一分×0.4」——新成员读测试就能理解模块行为。
3. **失败即 CI 阻断**：`npm run validate` / `pytest -q` 任一失败返回非零退出码，任何 Prompt / 规则 / 检索参数变更，先过测试再合入。

## 6.2 部署方案

| 方案 | 适用场景 | 成本 | 步骤 |
| --- | --- | --- | --- |
| GitHub Pages（在线演示 · 静态） | 免安装演示，随仓库更新 | 零成本 | `FM_API_BASE=<后端地址> node scripts/deploy-pages.js --build`（构建时注入后端地址，跨域访问） |
| 国内直连镜像（当前在线演示） | GitHub 访问慢时的备选；已由 FastAPI 后端同源托管 | 零成本 | `node scripts/build-backend-static.js` 后发布 `backend/`（演示页 + API + 文档 + 模型同一域名，免除 CORS） |
| Docker 容器（FastAPI） | 需要完整链路 + 自托管 | 自有服务器 | 基于 `backend/` 构建镜像，`uvicorn app.main:app --host 0.0.0.0 --port 3000` |
| Render / Railway（FastAPI） | 需要完整链路 | 免费额度 | `uvicorn app.main:app --host 0.0.0.0` |
| 本地演示 | 现场演示 | 零成本 | `python -m http.server 5173` + `uvicorn` |

**关键部署注意事项**

- **静态页必须压缩**
  ：单文件演示版 765KB，网关默认不压缩，后端已挂 `SelectiveGZipMiddleware`（实测 765KB → 220KB，−71%），并显式对 `/api/chat/stream` 跳过压缩——SSE 一旦被压缩，中间层缓冲会让逐字流式输出退化成整段到达；
- **必须用 http 服务打开，不能双击 `file://`**
  —— `file://` 的 Origin 是 `null`，会被后端 CORS 拒绝；
- `.env` 永不进仓库（`.gitignore` 已包含），Key 只存在于部署环境变量；
- 部署后要把地址回填到构建命令：`FM_API_BASE=<后端地址>`（Pages 跨域）或 `same-origin`（后端同源托管），README 顶部入口同步更新。

## 6.3 验证「真落库」

完成一次 5 轮面试后：

```
sqlite3 backend/futuremirror.db "select count(*) from turns"      # ≥ 5
sqlite3 backend/futuremirror.db "select id, mode, status from interviews"
sqlite3 backend/futuremirror.db "select interview_id, total from reports"
```

**这三条命令是回应「是不是前端仿真」质疑的最强证据**——数据在 SQLite 里，报告分和页面分一致。

## 6.4 快速启动

```
# 0. 克隆仓库
git clone https://github.com/Penguin-Zhou/FutureMirror.git
cd FutureMirror

# 终端 1：后端
cd backend && pip install -r requirements.txt \
  && cp .env.example .env \
  && uvicorn app.main:app --reload --port 8000

# 终端 2：前端
cd frontend && python -m http.server 5173

# 浏览器打开 http://localhost:5173（不要双击 index.html）
```

CHAPTER 07

# 核心算法详解与原创性论证

**本章意图**

本章把三个最容易被「AI 默认模板」作品含糊带过的模块——**多 Agent 调度**、**RAG 混合检索与重排**、**评分引擎**——展开到算法级：每一个都给出设计动机、参数、伪代码/公式、以及**可当场复现的校准数据**。这些细节不是装饰，是「这个作品为什么不是套壳」的核心证据。

## 7.1 多 Agent 协作调度算法

### 7.1.1 Agent 注册表：纯数据、双端共用

5 个 Agent 的人设集中在 `lib/ai/personas.ts`（纯数据模块，无副作用，客户端与服务端安全共用）：

| Agent | 考察重点 | 风格参数 | 调度层职责 |
|---|---|---|---|
| 💼 HR 面试官 | 动机 / 稳定性 / 协作 | 温和但有压迫感，连环追问动机 | 模式层固定位 + 内容层动机路由 |
| ⚙️ 技术面试官 | 项目深挖 / 原理 / 取舍 | 严谨，追问到底，要求量化数据 | 技术关键词路由 + 连环追问主体 |
| 🔥 压力面试官 | 抗压 / 边界 / 举证 | 直接尖锐，只对事不对人 | 压力模式主体 + 结构完整时触发 |
| 📝 观察员 Agent | 证据记录 / 风险标注 | 中立，只记录不提问 | 内容层派发的输入源（4 类标注） |
| 🎯 教练 Agent | 改进建议 / 学习路径 | 建设性、可执行 | 面试结束后汇总（报告页） |

### 7.1.2 两层调度：模式层轮转 + 内容层派发

调度不是一层，而是**两层串联**——先按面试模式确定轮转骨架，再按回答内容做单次派发决策：

| 层 | 输入 | 算法 | 输出 |
|---|---|---|---|
| **模式层** | 面试模式 + 轮次 turn | `pickSpeaker(turn, mode)`：O(1) 查表（comprehensive 按 [hr,tech,pressure] 轮转；tech 每 3 轮插入 HR；pressure 每 4 轮插入技术；gentle 恒定 HR） | 候选面试官 |
| **内容层** | 最近回答的关键词 + 观察员标注 | `planReason()`：技术词命中 → 技术面试官深挖；结构完整但缺量化/偏长 → 压力面试官边界追问；否则 → HR 考察动机 | 实际发言者 + **可解释理由** + 置信度 |

```
function planReason(speaker, keyword, notes, mode) {
  const k = keyword.slice(0, 12);
  if (speaker.id === "tech")
    return `技术关键词「${k}」${notes.includes("量化") ? " + 缺量化" : ""} → 技术面试官深挖原理`;
  if (speaker.id === "pressure")
    return `结构完整但${notes.includes("偏长") ? "篇幅偏长" : "细节待验证"} → 压力面试官做边界追问`;
  return `侧重动机协作 → HR 面试官考察稳定性`;
}
// 置信度：min(95, 60 + turn * 6) —— 轮次越多，画像越全，决策越自信
```

每次派发都会在面试间右侧时间线写一行**决策理由 + 置信度**，例如「📋 决策：技术关键词「Redis 缓存一致」 + 缺量化 → 技术面试官深挖原理（置信度 78%）」。**Agent 的每次「换人」都有据可查**——这是 Agent 决策过程的透明化，不是装饰。

### 7.1.3 连环追问：从「轮转」到「连续决策」

检测到技术关键词时，有 35% 概率触发连环追问——连续 2 轮深挖同一技术点，模拟真实技术面试官的追击：

```
function maybeStartDeepDive(keyword, mode, chance) {
  if (deepDive.left > 0) return { started: false };        // 已在追问中，不叠加
  if (isTechKeyword(keyword) && mode !== "pressure" && Math.random() < (chance ?? 0.35)) {
    deepDive = { topic: keyword, left: 2 };                // 锁定主题，再挖 2 轮
    return { started: true, left: 2 };
  }
  return { started: false };
}
```

固定题库轮转是「套壳」的典型特征；连环追问体现了 **Agent 根据上下文做连续决策**的能力，是 Agent 与「多轮问答机器人」的本质差异。

### 7.1.4 服务端同构调度与事件协议

FastAPI 路由 `/api/chat` 用同一套 `pickSpeaker` + 版本化 Prompt 模板（`prompts/interviewer.md`）组装系统提示，注入候选人画像（已确认技能 / 岗位缺口 / 匹配分），通过 SSE 事件协议向前端推送全过程：

| 事件 | 载荷 | 用途 |
| --- | --- | --- |
| `agent` | { id, name, title } | 前端实时显示「当前发言的是哪个 Agent」 |
| `thinking` | { text } | 「XX 正在分析你的回答…」占位，流式期间不冷场 |
| `delta` | { text } | 模型增量 token，逐字上屏 |
| `observer` | { note } | 观察员 Agent 的启发式批注（过短/缺量化/偏长/口头禅） |
| `done` | { text } | 完整回答落库，进入评分 |

隐私在调度层一并处理：只把**脱敏后（`maskPII`）的最近 8 条**对话发给模型。FastAPI 后端 `/api/chat/stream` 是同款代理（Key 只存服务端环境变量，前端零接触），线上已接通真实 GLM 流式（见 4.1）。

### 7.1.5 与「固定轮转模板」的差异

| 维度 | 模板作品常见做法 | 本作品 |
| --- | --- | --- |
| 发言者选择 | 固定题库轮转，每轮换人 | 模式层 + 内容层两层调度，按回答内容派发 |
| 决策可见性 | 无（用户不知道下一个为什么是它） | 派发理由 + 置信度实时写入时间线 |
| 上下文连续性 | 无状态轮转 | 连环追问状态机，锁定主题连续深挖 2 轮 |
| 人设工程 | 一个 system prompt 包打天下 | 5 个人设纯数据注册表 + 版本化模板 + 画像注入 |

## 7.2 RAG 混合检索与重排序策略

### 7.2.1 六阶段检索管线

检索不是「调一次 embedding API」，而是六个阶段、每阶段产出均可展示为证据的管线（端侧演示页 `frontend/js/data/kb.js` 与服务端 `lib/ai/rag.ts` 同策略实现）：

| 阶段 | 名称 | 实现 | 可展示证据 |
|---|---|---|---|
| ① | 分词 | 英文/数字整词 + 中文 bigram 切分 | 分词结果（开发者模式） |
| ② | 词面召回 | BM25：k1=1.5、b=0.75，标题字段重复计入 = 权重 ×2，IDF 恒正 | 逐条 bm25 分 |
| ③ | 语义召回 | MiniLM-L6-v2（384d，浏览器内推理）；无模型文件降级 n-gram hashing（256d，纯本地） | 逐条 cos 分 + 向量提供者 |
| ④ | 融合 | 两路各自 min-max 归一后加权：`0.6×cos + 0.4×bm25Norm`；纯语义召回设余弦门槛（MiniLM 0.15 / hashing 0.32） | 融合分 |
| ⑤ | **重排** | 对融合召回的 Top-8 候选用 5 个可解释词面/结构特征精排（见 7.2.6） | 重排分 + 五特征分解 |
| ⑥ | 引用构建 | Top-k 条目编号成上下文，答案必须标注来源，降低幻觉 | 引用列表 |

### 7.2.2 分词：为什么是 bigram 而不是接分词器

中文用户输入通常不带空格（「字节后端二面一般问什么」）。无分词器方案下，对含中文的 token 追加**二元词组（bigram）**是 CJK 检索的标准做法：「缓存穿透」切出「缓存 / 存穿 / 穿透」，召回「Redis 缓存穿透/击穿/雪崩区别」条目的同时，单字词被过滤（长度 < 2 丢弃）防噪声。零依赖、毫秒级、与服务端策略一致。

### 7.2.3 为什么不是「纯向量检索」

| 方案 | 优点 | 缺点 | 结论 |
|---|---|---|---|
| 纯向量检索 | 语义泛化好 | 关键词精确命中差（「B+ 树」可能召回「二叉树」） | ❌ |
| 纯 BM25 | 关键词精确、快 | 同义改写召回差 | ❌ |
| **混合 + 重排** | 两路互补 + 词面特征精排 | 需处理归一与权重 | ✅ **采用** |

### 7.2.4 端侧语义通道与四级降级

MiniLM 权重（23MB）的获取链路按国内网络实测排序：**① 站点同源 models/ → ② GitHub Pages models/（带 ACAO:*）→ ③ hf-mirror.com → ④ huggingface.co**；运行时库 similarly 多 CDN 备选（fastly 节点国内实测最快），单源 10s 超时即切换。全部不可达时降级 hashing 向量（门槛抬高到 0.32，因为它没有真实语义能力），**检索链路不中断**。40 条向量预计算一次写入 localStorage，二次访问零推理。

### 7.2.5 加权融合

两路分数各自 min-max 归一后加权：`score = 0.6 × cos + 0.4 × bm25Norm`。余弦权重更高——语义通道解决「换一种说法」的召回，词面通道保证精确命中；纯语义召回的条目须过余弦门槛，防止 hashing 兜底时噪声混入。

### 7.2.6 五特征重排：召回管「相关」，重排管「排序」

**动机**：加权融合隐含假设「余弦高」与「词面准」等价重要，但面试场景里「标题命中查询意图」几乎总是更强信号——用户搜「MySQL 索引」，标题含这四个字的条目应排在只在正文提及的条目前面。重排阶段把这些可解释特征显式建模：

| 特征 | 权重 | 定义 | 直觉 |
|---|---|---|---|
| title | 0.34 | 查询词（含 bigram）在标题中的覆盖率 | 标题最能代表条目意图 |
| tag | 0.22 | 查询词在标签中的覆盖率 | 标签是人工凝练的主题词 |
| phrase | 0.24 | 查询与标题/正文的最长公共子串占比（标题 ×1.0，正文 ×0.6） | 完整短语 = 强意图信号 |
| type | 0.12 | 条目类型先验：面经 1.0 / JD 0.95 / 题库 0.9 / 指南 0.85 / 报告 0.8 | 问「怎么问」时面经优先于趋势报告 |
| len | 0.08 | 长度先验：距理想长度 170 字越近越高 | 过短信息量不足，过长稀释密度 |

```
final = 0.55 × hybridNorm + 0.45 × featScore
// hybridNorm：候选池内混合分归一；featScore：五特征加权和（权重和 = 1）
```

**为什么不用 LLM rerank**：延迟（多一次模型往返）、成本、以及最关键的——**确定性。**规则重排对同一查询永远给出同一结果，特征分可逐项展示给用户核对；LLM 重排三者皆无。40 条规模的知识库，规则重排的区分度已经够用。

**重排真实效果**（线上知识库实测，召回序 → 重排序）：

| 查询 | 混合召回 Top-3 | 重排后 Top-3（变化） |
|---|---|---|
| MySQL 索引 | B+ 树条目 / 后端 JD / 事务隔离级别 | B+ 树条目 / **事务隔离级别** ↑ / **后端 JD** ↓（标题零命中，phrase 0.60 也救不回） |
| Redis 持久化 | RDB 与 AOF / 分布式锁 / 缓存三兄弟 | RDB 与 AOF / **缓存三兄弟** ↑ / **分布式锁** ↓ |
| 后端 开发 工程师 | 后端 JD / AI 求职趋势 / 腾讯一面 | 后端 JD / **腾讯一面** ↑ / **字节二面** ↑ / **AI 趋势** ↓ |

RAG 页对每条引用逐条展示三阶段得分：`bm25 2.31 · cos 0.44 · 融合 0.40 · 重排 0.91`——**检索过程完全透明，访客可以当场核对每一个数字怎么来的。**

### 7.2.7 双档语义通道：同一套策略，两种工程取舍

| 档位 | 语义通道 | 运行环境 | 取舍理由 |
|---|---|---|---|
| 端侧语义增强版（可选） | all-MiniLM-L6-v2 384d | 浏览器（Transformers.js + wasm，权重按需从 CDN 加载并缓存 localStorage） | 真实语义编码能力，首次加载需联网，之后离线可用 |
| 端侧离线版（默认） | n-gram hashing 256d 稀疏向量 | 浏览器（纯 JS，零依赖零网络） | 离线可跑，hashing 保证零依赖确定性，融合 / 重排策略与语义版逐位一致 |

两套档位都在**浏览器内**运行，共享同一套 BM25 + 余弦融合 + 五特征重排策略；区别仅在语义向量的来源（真实模型 vs. hashing 兜底）。后端仅负责鉴权、持久化与 LLM 流式代理，不承载检索，因此演示页与 FastAPI 同源部署即可，无需独立的检索服务。

## 7.3 评分引擎设计与校准过程

### 7.3.1 设计目标

评分引擎要回答四个问题：① **可解释**（每个分数能追溯到对话原话）；② **可复现**（同一输入永远同一输出，不靠模型运气）；③ **双引擎同构**（前端本地引擎与服务端引擎逐位一致，支撑三级降级）；④ **对匹配报告开放**（简历匹配分作为先验进入拟合维度，而非另起一套）。规则引擎（而非让 LLM 直接打分）是同时满足四者的唯一选择。

### 7.3.2 五维信号设计

| 维度 | 权重 | 评分信号（可机械判定） | 满分条件 |
|---|---|---|---|
| 内容深度 | 25% | 数字个数（×2.2 上限 22）、技术词命中（×2.5 上限 18）、轮次 ≥4 +8、代码沙盒通过率（×0.8/0.2 融合） | 4+ 量化且 3+ 技术词 |
| 结构逻辑 | 20% | STAR 词命中（背景/目标/负责/通过/最终/结果/因此/于是）×4 上限 30 | 命中 3+ |
| 语言表达 | 15% | 口头禅（然后/那个/就是）×5 上限 24、短答数（<40 字）×4 | 口头禅 ≤2 且无短答 |
| 岗位匹配 | 20% | 匹配报告分 ×0.4 + 技术词命中（上限 10）；无报告时按技能命中估算 | 匹配分高 + 缺口少 |
| 抗压表现 | 20% | 防御性表述（不是我的/跟我没关系/但是/我没有）×6 上限 20；压力轮短答 ×3 上限 12 | 无防御性表述 |

### 7.3.3 边界裁剪的设计理由

所有维度分裁剪到 `[30, 96]`：**下限 30**——再差的回答也有保底，避免「零分挫败」吓退用户；**上限 96**——不留满分，保留「还能更好」的成长空间，也与学习路径的「补强」叙事一致。总分 = 五维均值（半价向上舍入）。

### 7.3.4 校准过程：四步，含一次真实缺陷修复

1. **规则来源**：信号不是拍脑袋——量化数据（STAR 面试法的 Result 可验证化）、STAR 结构词、口头禅密度、防御性表述，分别对应面试评估中内容、结构、表达、抗压四类公认维度。

2. **双引擎对拍**：`backend/scripts/calibrate_scoring.py`（服务端）与 `scripts/calibrate-scoring.js`（前端 jsdom）共用同一份标准样本 `calibration-cases.json`（5 个答题质量递进的样本），逐维度比对。
3. **发现并修复舍入分歧**：对拍首跑发现 C1/C5 的「内容深度」双引擎差 1 分。根因：**Python 内置 `round()` 是银行家舍入（90.5 → 90），JS `Math.round` 是半价向上（90.5 → 91）**——评分真值恰好落在 .5 边界（48 + 22 + 5×2.5 + 8 = 90.5）。修复：服务端改用 `floor(v + 0.5)` 与前端逐位一致。**对拍从 3/5 变为 5/5 完全一致**。这个缺陷靠肉眼读代码发现不了，是对拍方法论的价值证明。
4. **不变量固化**：单调性（答题质量提升 → 总分提升）、维度分离（每个维度只对目标特征敏感）、确定性（重复评分一致）、边界（全部落在 [30,96]）写成 `backend/tests/test_calibration.py`，11 条断言常驻回归。

### 7.3.5 校准数据（双引擎一致输出）

| 标准样本 | 总分 | 内容深度 | 结构逻辑 | 语言表达 | 岗位匹配 | 抗压表现 |
|---|---|---|---|---|---|---|
| C1 量化密集的技术强答 | **77** | 91 | 78 | 84 | 65 | 66 |
| C2 结构尚可但缺量化 | **64** | 48 | 74 | 76 | 55 | 66 |
| C3 短答 + 口头禅 | **55** | 48 | 54 | 52 | 55 | 66 |
| C4 压力面防御性表述 | **56** | 48 | 58 | 76 | 55 | **43** |
| C5 C1 强答 + 匹配报告与代码分 | **82** | 90 | 78 | 84 | **92** | 66 |

读表方式：**总分单调**（55 → 56 → 64 → 77 → 82）；**每个维度只对目标特征敏感**——量化数据把内容深度从 48 拉到 91、STAR 结构把结构逻辑从 54 拉到 78、口头禅把语言表达压到 52、防御性表述把抗压表现孤立到 43（其他维度不动）、匹配报告把岗位匹配推到 92。**区分度即校准证据。**

### 7.3.6 证据链与学习路径生成

报告页每条证据显示 `[Agent 名称] "引用你的原话"` 观察员标注（取最后 4 轮）——分数不是黑箱。学习路径按最弱维度动态分配天数（3/2/1 + 2 天基础项），用户知道为什么第 1-3 天要练这个。

## 7.4 与「AI 默认模板」作品的差异论证

| 模块 | 模板作品常见做法 | 本作品做法 | 证据位置 |
|---|---|---|---|
| 多 Agent | 单 Agent + 固定题库轮转，或只会在文档里画架构图 | 两层调度（模式层 + 内容层）+ 可解释派发 + 连环追问状态机 + SSE 事件协议 | 7.1；面试间时间线实物 |
| RAG | 调一次 embedding API，或纯向量检索，结果不可解释 | 六阶段管线：bigram 分词 → BM25 → 双通道语义 → 加权融合 → 五特征重排 → 强制引用 | 7.2；RAG 页三阶段得分 |
| 评分 | 让 LLM 直接打分（黑箱、不可复现、每次不同） | 规则引擎 + 双引擎对拍 + 舍入缺陷修复 + 单调性测试 + 证据链 | 7.3；校准表与 11 条断言 |
| LLM 集成 | 前端直连 Key（泄露风险）或纯 Mock 假界面 | 服务端代理（Key 零前端接触）+ 三级降级 + 线上真实流式已验证 | 4.1；线上 /api/health |
| 工程完成度 | 能跑演示就行 | 2026 前端断言 + 44 后端测试 + 双引擎对拍 + 四渠道发布流水线 | 第 6 章 |

**差异的共同点**

每一个设计选择都绑定一个

**具体工程问题**

（中文无空格 → bigram；国内网络 → 四级权重源；评分不可复现 → 规则引擎 + 对拍；.5 边界 → 舍入规则统一），并且都有

**可当场验证的证据**

（时间线、三阶段得分、校准表、对拍脚本、测试断言）。模板作品的特征恰好相反：做法来自教程默认值，证据只有截图。

## 7.5 技术创新点

| # | 创新点 | 具体实现 | 为什么是创新 |
|---|---|---|---|
| **T1** | 可解释的多 Agent 派发 | 派发理由 + 置信度实时写入时间线 | 常见 Agent 系统只做派发，不解释「为什么派给这个 Agent」 |
| **T2** | 连环追问机制 | 技术关键词触发 35% 概率连续深挖 2 轮 | 从「轮转」升级到「连续决策」，是 Agent 与机器人的分水岭 |
| **T3** | BM25 + 端侧向量混合检索 + 五特征重排 | 参数调优 + Transformers.js + 融合权重 + 重排精排 + 离线兜底 | 端侧语义检索的完整工程实现，不是调 API |
| **T4** | 逐条展示三阶段得分的 RAG 引用 | `bm25 2.31 · cos 0.44 · 融合 0.40 · 重排 0.91` | 检索过程完全透明，可当场核对 |
| **T5** | 可解释评分 + 证据链 | 每条分数引用对话原话 + 观察员标注 | 评分不是黑箱，每条都能追溯 |
| **T6** | 前后端评分引擎同构 | Python 版与 JS 版算法一致，交叉验证 | 三级降级在评分层的具体实现 |
| **T7** | MCP 工具运行时可验证 | 开发者模式内置 MCP 控制台 | 支持 MCP 的产品很多，能让访客当场验证调用链路的少 |
| **T8** | 代码沙盒入评分 | Web Worker + 2s 熔断，通过率并入内容深度维度 | 把「编码能力」这个难量化的维度变成可量化信号 |
| **T9** | 隐私优先的视觉状态监测 | 前端 face-api.js 本地识别在场/专注，仅上报结构化结果；后端聚合在场率/专注率 | 把「摄像头」从隐私风险点变成可解释的实时状态信号，原始帧不出浏览器 |

## 7.6 应用创新点

| # | 创新点 | 说明 |
|---|---|---|
| **A1** | 求职全流程闭环 | 简历匹配 → 简历优化 → 面试 → 报告 → 题库 → 求职看板，形成闭环而非单点工具 |
| **A2** | 题库 + 错题本 + 每日一练 | 把「背八股」变成可量化的学习过程 |
| **A3** | 职业探索匹配引擎 | 按专业 + 技能 + 兴趣匹配 24 个岗位，给出技能对比与学习路径 |
| **A4** | AI 小助手（全局悬浮） | 可拖动贴边冻结，检索知识库作答，使用引导一键直达 |
| **A5** | 开发者模式（面向访客） | 把「技术透明度」做成一个面向访客的产品功能，而不是文档里的一段说明 |
| **A6** | 移动端体验优化 | 顶部导航、触控 ≥42px、输入框 16px 防 iOS 缩放、底部安全区适配 |

## 7.7 创新点总结

**最大的创新不在单点技术，而在「可解释」这个设计哲学贯穿全产品**

- Agent 派发可解释（为什么派给他）；
- RAG 检索可解释（两路得分怎么算）；
- 评分可解释（为什么是这个分）；
- 学习路径可解释（为什么练这个）；
- 降级可解释（为什么现在是 Mock）。

**这种「全链路可解释」在 AI Native 应用里是稀缺的**

——大多数产品追求「更智能」，但用户不知道智能从哪来。面镜反过来，每一步都留下可核查的证据。

CHAPTER 08

# 总结与展望

## 8.1 已完成

- **5 个 Agent 协作 + 动态派发 + 连环追问**的完整面试引擎；
- **BM25 + 端侧向量混合检索**的 RAG 层，含缓存、降级、可解释得分；
- **可解释评分 + 证据链 + 动态学习路径**的报告引擎；
- **三级降级**的完整链路（后端 → 直连 → Mock），演示不中断；
- **2393 + 57 项自动化测试**，覆盖所有功能；
- **开发者模式**作为面向访客的技术透明度工具。

## 8.2 已知边界

| 边界 | 说明 |
| --- | --- |
| 演示版向量库 | 浏览器内 localStorage 单机索引（40 条），非分布式向量数据库 |
| 端侧模型 | 首次加载需下载 ~23MB；纯离线环境降级为 hashing 向量（仅词面） |
| MCP | 本地模拟 Server，协议一致但非跨进程真实 MCP |
| 代码沙盒 | Web Worker 隔离，仅支持纯 JS 逻辑题 |
| 登录系统 | 演示级（本地混淆），非生产级认证 |
| 数据库 | SQLite 单机，生产需替换 PostgreSQL |

## 8.3 未来规划

| 方向 | 计划 |
| --- | --- |
| RAG 升级 | pgvector 持久化 + RRF 融合 + Cross-Encoder Rerank |
| Agent 升级 | 真实 MCP Server + 跨 Agent 工具发现 + 多轮规划 |
| 评分增强 | 在已落地的确定性引擎 + LLM 质性增强基础上，引入人工专家复核样本量化分维度偏差（替代黑箱打分） |
| 多模态 | Whisper ASR + 语速 / 停顿分析 + 白板代码识别 |
| 部署 | 多租户隔离 + 限流 + PostgreSQL + 对象存储 |
| 商业模式 | 院校就业中心合作 / 企业校招 SaaS / C 端订阅 |

## 8.4 致谢与声明

本项目开发过程中使用 AI 编程工具作为辅助手段，**CSS 主题样板、通用工具函数、测试用例骨架、部分页面文案由 AI 辅助生成**；**多 Agent 角色设计与派发策略、可解释评分规则与证据链、RAG 检索层参数调优与融合策略、业务架构与前后端契约、安全方案、测试体系均为团队独立设计与实现。**所有 AI 生成内容均经过团队逐行审查、修改与测试，完整 Git 提交记录可见演进过程。演示数据全部为人工合成示例，无爬取真实网络数据。

CHAPTER 09

# 版本更新日志（v1 → Beta 1.1.2）

本日志为**团队真实开发进程**的版本记录，覆盖 2026.08（v1 初始原型）→ 2026.10（Beta 1.1.2 当前版本），共 35 个已发布版本。每个版本均可独立运行、可回归。

**关于 Git 提交记录（重要说明）：**

本仓库的 Git 提交历史自

**Beta 版本**

（首次以 `v1.0.0-beta` 提交包上传）时建立，

**仅包含 BETA 1.0 及之后的更新**

；BETA 1.0 之前的 v1–v22 演进历程以本更新日志形式完整留存，未在 Git 提交记录中逐条提交。日志中所载为团队真实的版本演进，而非事后补造。

阅读指引：本日志按版本倒序排列。带「当前版本」标记的是 Beta 1.0.9，为当前正式发布版本（专家能力层：主题跟随系统 + 印刷风质感 + 微交互 + 滚动驱动 + 数据可视化 + 高级组件）；带「规划中」标记的版本为下一步迭代计划，尚未进入代码库。所有数据指标均来自对应版本代码库的实际统计；仅记录团队自研功能演进，不含任何第三方内容。

## 版本摘要（36 个版本：35 个已发布 + 1 个规划中）

完整逐版本明细（新增功能 / 优化改进 / 数据指标）见在线更新日志页 `changelog.html` 与仓库 `changelog/content.js`；本表仅为打印版摘要。

| 版本 | 日期 | 主题 | 代表性内容 |
| --- | --- | --- | --- |
| v23（规划中） | 待排期 | 规划中 · 生产能力对齐 · 生产级 RAG · 多模态扩展（待排期） | pgvector 持久化：向量库从浏览器 localStorage 迁到 PostgreSQL + pgvector，支持跨设备与多用户 |
| Beta 1.0 | 2026.10 | 检索层升级与版本定位 | BM25 词面召回：k1=1.5、b=0.75，IDF 用 log(1+…) 保证恒正，标题字段重复计入 = 权重 ×2；中文 bigr |
| Beta 1.0.1 | 2026.10.20 | 后端上线 · 全渠道接通真实服务 | 后端同源托管：backend/static/（演示页 + 技术文档 + 更新日志 + 41MB 端侧模型）由 FastAPI 挂载到站点 |
| Beta 1.1.2 | 2026.10.30 | 体验修复层 + 增量批次 v33–v50：缺陷修复 / 学习计划 / 图标 / 摄像头状态 / 求职闭环 / 可视化 / 反馈闭环 / 移动端裁剪 | 模拟面试无法重新面试：重面入口此前只藏在 ⌘K 面板（清空对话记录）与报告页（再来一次），面试页 chat-head 新增「🔄 重新面 |
| Beta 1.1.1 | 2026.10.23 | 审计加固层：全站与仓库逐项复核后的收口 | 文档一致性：技术文档 <title> 长期停留在 v1.2（正文已是 v1.9）→ 修正为 v1.10 并随版本联动；补 & |
| Beta 1.1.0 | 2026.10.23 | 完整性补全层：把能力清单的缺口逐项封死 | 骨架屏（skeleton.js）：mount(host, opts) 生成 .bp-skel 占位块（同容器幂等），clear(forc |
| Beta 1.0.9 | 2026.10.22 | 专家能力层：把高级网页能力清单逐项补齐 | 主题三态（theme.js + index.html 首屏预置）：themeMode() / setThemeMode(m) / cyc |
| Beta 1.0.8 | 2026.10.22 | 深度交互层：让每个动作都有回音 | 命令面板 Pro（command.js）：最近访问分组（loadRecent/saveRecent/recordRecent，经 win |
| Beta 1.0.7 | 2026.10.22 | 全站交互层：把图纸动作铺到每一个页面 | 切页编舞：showView / applyRoute 挂钩 choreograph()——标题区、卡片按 --bp-i 索引 stagg |
| Beta 1.0.6 | 2026.10.22 | 交互深化层：命令面板与滚动语言 | 命令面板（command.js）：⌘K 懒建注入，分组渲染（导航 4 / 页面 10 / 动作 3）、子串过滤、↑↓ / Enter / |
| Beta 1.0.5 | 2026.10.22 | 动态层：粒子场与微交互 | 蓝图粒子场（particles.js）：DPR 适配（上限 2x）、粒子数按视口面积封顶（30–90）、rAF 驱动 + 被动监听、主题 |
| Beta 1.0.4 | 2026.10.22 | 体验层增强 · 交互反馈与可访问性 | 打字指示器：.typing-dots 三点墨点脉冲 + 等宽标签，真实 LLM 流式与 Mock 打字机双通道接线（首字到达即接管），失 |
| Beta 1.0.3 | 2026.10.21 | 蓝图编辑风视觉重构 | 蓝图编辑风设计系统：theme.css 全量重写——纸 / 墨 / 线 / 朱红四组令牌、暗色主题同构、衬线 + 等宽 + 无衬线三字体 |
| Beta 1.0.2 | 2026.10.21 | 原创性论证与真实度提升 | 第 7 章「核心算法详解与原创性论证」：三大算法逐一展开——多 Agent 圆桌协作的派发调度、RAG 混合检索 + 五特征重排（fin |
| v22 | 2026.10 | 后端适配与工程化 | FastAPI 后端：backend/ 目录，含 db.py / llm.py / scoring.py / main.py，附 .en |
| v21 | 2026.10 | 悬浮球拖拽冻结 + 移动端美化 | 悬浮球拖拽：pointer 事件接管拖动（touch-action: none），位移阈值 6px 抑制误触；拖动中按实时位置判定贴边 |
| v20 | 2026.10 | 题库增强 + AI 小助手 + 移动端导航顶部化 | 错题本（VIP）：自动收录答错与超时的题目，支持按分类筛选、重复练习、移除已掌握；持久化到 fm_wrong |
| v19 | 2026.10 | 题库小测模块 + VIP 会员体系 | 题库页（quiz）：新增第 10 个功能页，含 3 个子页签（知识点学习 / 限时小测 / 成绩档案） |
| v18 | 2026.10 | 产品闭环深化 · 求职管理 + 面试真实感 | 求职管理页（jobs）：新增第 9 个功能页，含 2 个子页签 |
| v17 | 2026.10 | 深海蓝 AI Native 视觉重构 | 深海蓝主题：暖白背景 #FAFBFF + 深海军蓝文字 #10163A + 电光蓝品牌色 #2E3FE8 |
| v16 | 2026.10 | 个人中心 · 头像系统 · 偏好与数据管理 | 个人中心页（profile）：新增第 8 个功能页 |
| v15 | 2026.10 | Beta · 工作台精简与互动升级 | 今日推荐岗位：从 24 个岗位中随机抽取 4 个，点击直达模拟面试，支持"🔄 换一批" |
| v14 | 2026.10 | 开发者模式升级 | 运行环境检测：自动识别浏览器 / OS / 语言 / 网络 / 屏幕 / 视口 / DPR / CPU 核心 / 设备内存 |
| v13 | 2026.09 | 验证专区（加密 + 一键进入） | 验证专区视图：#/evaluator?key=FM2026 独立视图 |
| v12 | 2026.09 | 可验证运行时能力三件套 | MCP 实时控制台：3 个工具（search_job_db / fetch_interview_experience / schedul |

| 版本 | 日期 | 主题 | 代表性内容 |
| --- | --- | --- | --- |
| v11 | 2026.09 | 目标岗位下拉选择器 | 岗位下拉选择器：5 个分组（技术 / 产品 / 设计 / 商业 / 新兴）共 24 个岗位 |
| v10 | 2026.09 | 导航重构与数据扩充 | 左侧栏导航（桌面）：240px 固定宽度，2 个分组 + 7 项 + 面试进度徽章 |
| v9 | 2026.08 | 修复 HTML 结构 + 细节完善 | app-page 嵌套错乱：把 6 个非 dashboard 的 app-page 从 dashboard 中移出，做成平级兄弟元素 |
| v8 | 2026.08 | 多页面路由 + 导航系统 | hash 路由系统：7 个页面 #/dashboard / #/match / #/interview / #/rag / #/resu |
| v7 | 2026.08 | 首页落地页 + 游客模式 | 首页落地页：Hero + 产品预览 Mock + 功能网格 + 工作流 + 数据条 + CTA |
| v6 | 2026.08 | 真实模型修复 + 新板块 + UI 重构 | 简历优化工作台：6 维诊断（量化 / STAR / 动词 / 匹配 / 精炼 / 技能）+ 逐条改写建议 + LLM 深度改写 |
| v5 | 2026.08 | 登录系统 + UI 全面升级 | 登录 / 注册页：左右分栏布局，左侧品牌 + 特性，右侧表单 |
| v4 | 2026.08 | 深度完善版 | 实时能力追踪器：面试过程中动态更新雷达图，显示进度 / 字数 / 量化 / 置信度 |
| v3 | 2026.08 | 深度完善 · 修复模型调用 | 简历优化工作台：6 维诊断 + 逐条改写建议 |
| v2 | 2026.08 | 体验完善版 | 暗色模式：data-theme="dark"，localStorage 持久化 |
| v1 | 2026.08 | 初始原型 | 简历匹配：粘贴简历 + JD，启发式匹配度计算 |

Beta 1.0.1 为 Beta 1.0 的部署补丁：FastAPI 后端正式上线（国内镜像域名同源托管演示页 + API，GitHub Pages 构建时注入后端地址跨域接通），功能集与版本号展示不变；未配置模型 Key 时聊天自动回退 Mock，演示不中断。